

Defect Generation Using Inpainting Stable Diffusion

Minh Le

[GitHub Repo](#)

1. Introduction

Before starting my Master's degree, I worked as a Computer Vision Engineer at an AI start-up developing automated defect inspection systems. Our work involved building models for defect classification and controlling peripherals to remove defective products from production lines. Like most AI applications, these systems require large amounts of high-quality training data. However, collecting and manually creating defect samples was often time-consuming and labor-intensive. Motivated by this challenge, I aim to develop an AI model capable of generating realistic synthetic defects from only a few defect samples.

The approach that I used in this assignment follows J.Song, et.al research [1] which will be referred as *DefectFill*. The idea is to fine-tune an inpainting diffusion model using just a few samples with LoRA to reduce training time and GPU resources. What sets DefectFill apart from contemporary methods is its custom loss function (a combination of 3 separate loss functions) allowing the model to have a precise knowledge of the defect features and realistically integrate it into the defect-free object.

2. Literature Review:

The earliest research in defect generation field by [2] made use of segmentation techniques to extract defects on a few annotated samples which are then cropped and pasted onto a defect-free sample. When the defects are added to defect-free samples, they introduced a weighted loss to enhance blending between defects and objects. However, this method is still limited to using in-distribution defects without introducing any new synthetic defects.

Later on, generative AI models are introduced to defect generation applications. DFMGAN [3] incorporates defect-aware residual blocks to GAN backbone to produce defect masks and work only within this region. However, DFMGAN requires long training time and GAN-based methods are often prone to mode collapse.

Researchers also made use of Stable Diffusion to generate defects, one promising research was AnomalyDiffusion [4]. Adaptive Attention Re-weighting Mechanism allows the model to focus on regions where the defect is barely visible and make future generations better match their masks. This approach however, still struggles in linking the relation between defects and object, resulting in unnatural defects.

DefectFill

The main idea of the approach is to give a stable diffusion inpainting model a comprehensive knowledge of a defect by associating a word token [V*] to a specific defect region. This token is different from transformer's tokens, in which they encode the visual concept of a defect class, instead of an actual word. This concept is called textual inversion [5]

Most notably, the authors introduce a weighted sum of three losses:

- Defect loss captures the fine details of the defect itself.
- Object loss learns the relationship between the defect and the object, ensuring that defects blend naturally with the surrounding context.
- Attention loss provides a spatial guidance mechanism, showing the model where to look and not just what to generate.

The total loss is a linear combination of three individual losses:

$$\mathcal{L}_{ours} = \lambda_{def} \cdot \mathcal{L}_{def} + \lambda_{obj} \cdot \mathcal{L}_{obj} + \lambda_{attn} \cdot \mathcal{L}_{attn}.$$

Where λ is the weight for each term. My settings also follow that of the paper with the weights being 0.5, 0.2 and 0.05. The dominant λ_{def} also puts more weight on learning small defects reflecting the primary objective which is realistic defect texture.

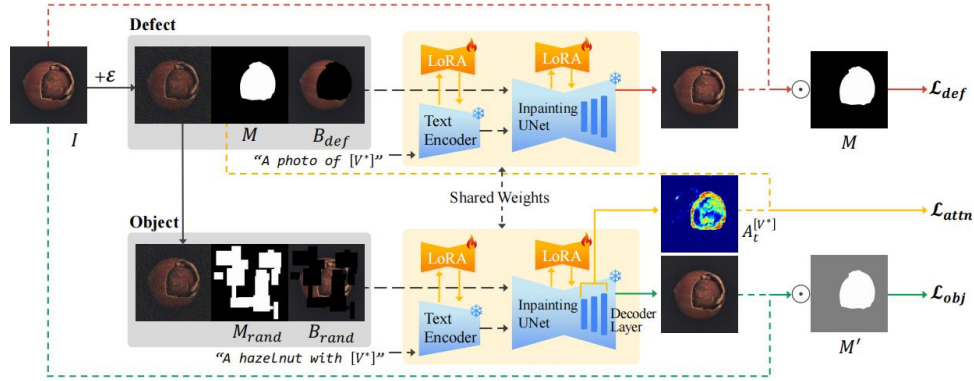


Figure 1. Defect Fill model's architecture.

Full fine-tuning a diffusion model is computationally prohibitive, thus, this approach uses LoRA. The model's backbone is frozen and fine-tuned by using a subset of LoRA parameters with a rank of 8. LoRA modifies the attention projection matrices in UNet and CLIP text encoder. However, the token embedding layer is made fully trainable as it needs more expressivity than a low-rank update allows.

One major drawback of DefectFill is that it is fine-tuned per object class, in other words, it is not a universal defect generator. Also, the model depends heavily on mask quality at both training (for loss computation) and inference time (to define where the defect is generated). Therefore, DefectFill is only applicable when the exact location of the defect is defined (with masks). In practice, deciding realistic defect placement on a clean image is a non-trivial problem. A poorly placed mask produces an unrealistic synthetic defect regardless of generation quality.

3. MVTec Dataset:

MVTec AD is a real-world industrial anomaly detection benchmark containing 15 object categories spanning both textures (carpet, leather, tile) and structural objects (pill, screw, transistor). The dataset

provides paired clean and defective images along with pixel-level ground truth masks, making it suitable for DefectFill's mask-conditioned inpainting framework.

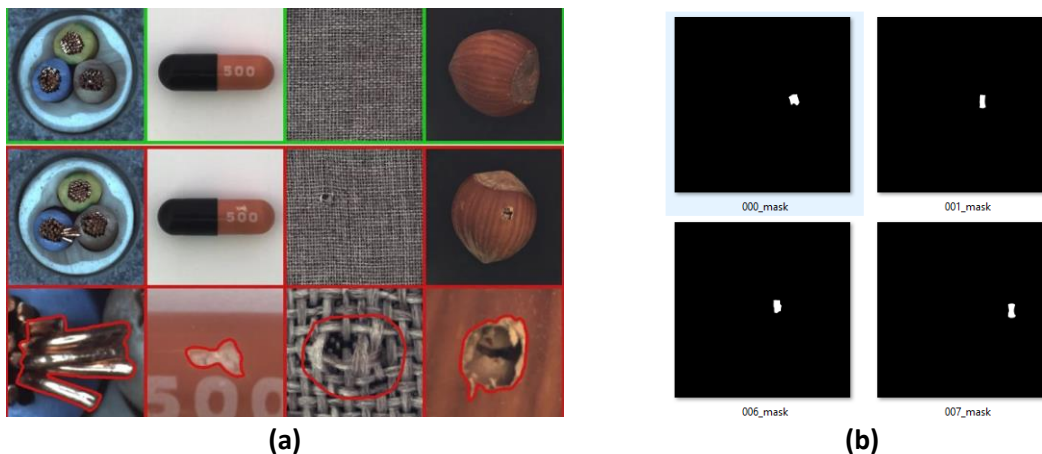


Figure 2. (a) Sample images from MVTec AD dataset (b) Defects' masks.

Although the dataset contains 5000 images, only ~1260 images contain defects while the remaining images are masks and defect-free samples. Figure 3 shows notable imbalance across object categories where the number of defects for most represented objects (pill – 141, screw – 119, zipper – 119) is more than 2 times higher than the least represented (grid – 57, transistor – 40, toothbrush – 30). Since the models are fine-tuned per object class, objects with fewer samples will likely produce lower quality generations.

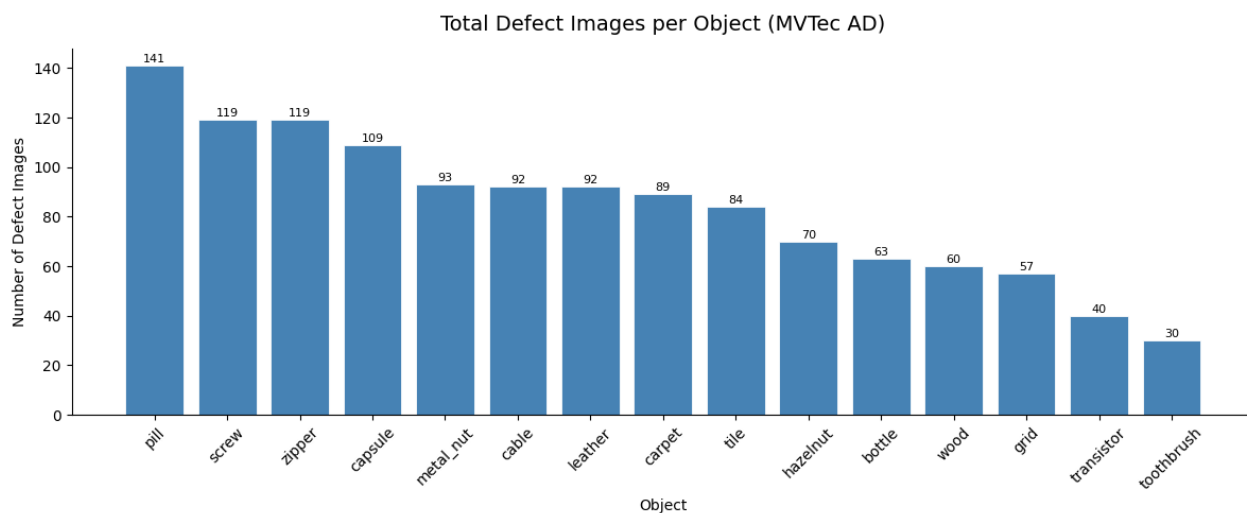


Figure 3. Defect count per object.

Since most defect types are unique to specific object categories (Figure A1, Appendix), the motivation for a unified multi-object, multi-defect model is not immediately compelling, even though defects such as color variations, scratches, and cracks appear across multiple categories. In industrial settings, end users typically focus on a single product type, making object-specific models a more practical and cost-

effective solution. Furthermore, the visual characteristics of the same defect can vary substantially across materials. For example, a scratch on leather may look very different from a scratch on metal.

4. Implementation

The code implementation is hugely inspired by axelsig1 [6] – an unofficial implementation of DefectFill. I extended the training and inference logic to adapt the model to multi-defect problem.

Due to time constraints, only two objects: bottle and leather were used to train defect generation models. Apart from two vanilla models for each type of object, three other models were trained for ablation study, each with different loss term being ablated.

A major limitation of the current implementation is the extensive use of hard-coded parameters, largely due to the notebook-based development environment. Whenever a configuration changes, parameters must be manually updated across multiple cells rather than being managed through centralized configuration files, as would be typical in a standard software project. In addition, the training, inference, and evaluation pipelines are implemented within a single notebook, requiring sections of code to be commented out depending on the task being performed. While this design is not ideal from a software engineering or production perspective, it reflects the workflow commonly adopted in Kaggle and Google Colab environments. On the positive side, keeping the code, experiments, and results in a single notebook improves reproducibility and makes the workflow easier to follow and interpret.

Memory Complexity

On the first time when I ran the training loop, I ran out of GPU memory with 14.5GB of VRAM used. The majority of the memory came from Stable Diffusion SD2-inpainting model with UNet (~860M params), CLIP ViT-H text encoder (~354M params), VAE (~84M params). To resolve this issue, I offload VAE to CPU after use to reduce the strain on GPU's memory at the trade-off being a transfer latency each time VAE is reloaded onto GPU (happens 3 times in the training loop).

Computational Complexity

An image passed through the VAE encoder has its spatial dimensions downsampled to $64 \times 64 = 4096$ tokens. In self-attention, every token attends to every other tokens, resulting in an attention matrix with 4096×4096 which is approximately ~16M entries per head per layer for each UNet pass. With 50 denoising steps during inference, that is 50 UNet forward passes.

5. Results

Figure 4 shows the training loss on *bottle's* defects for reference; the raw loss is expected to be noisy as it is sampled per sample and further amplified by a small batch size (2 images per batch) due to limited GPU resource. The smoothed version, however, shows a general decline in loss value across all 4 losses. Object and total loss show the cleanest convergence curves, dropping sharply in the first ~150 steps before stabilizing. Attention loss oscillates around 0.135 – 0.145 with minor decline over 500 steps, mainly due to its smaller weights compared to the other two terms (10x and 4x less than defect and object losses respectively). The total loss is heavily dominated by defect and object gradients, resulting in

attention barely being optimized. Defect loss shows the opposite, while it drops fast early (~ 1.05 to ~ 0.7) it bounces back from the 150 step onward, suggesting that the model is not fully converged and training for 500 steps is not sufficient (the authors trained for 2000 steps).

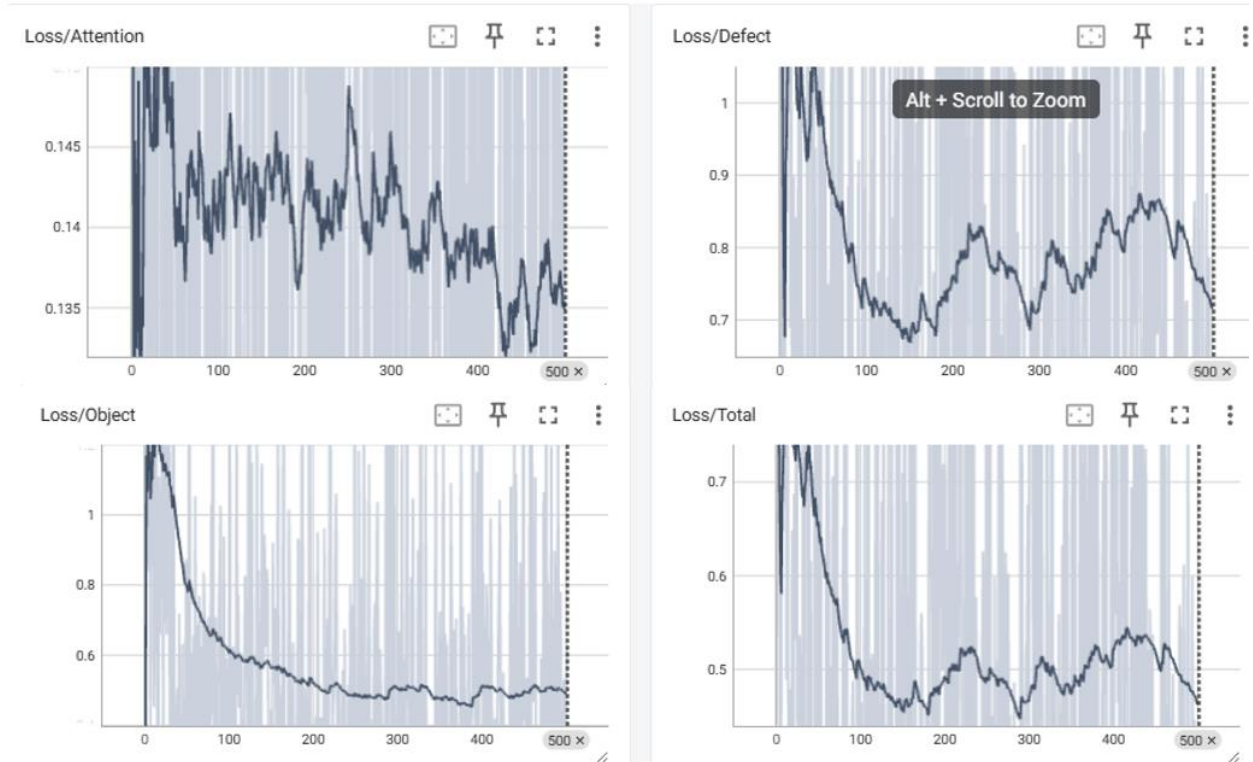


Figure 4. Training losses for bottle class.

Qualitative Results

Figure 5 shows the generated defects on bottles and leather with a small binary image representing the mask at the lower right corner of each sample. Visual inspection reveals that the model performs unevenly across defect types. The result for *broken_large* looks convincing, as the silver, glistening glass texture blends naturally with the bottle interior. *broken_small* shows a reasonably small chip although it is overly sharp at the edges, also, the prominent green color makes it a less realistic generation. Contamination is the weakest result, with the foreign object (likely a literal bottle) appearing visually inconsistent with the surrounding material. The colour, texture, and boundary sharpness all suggest a compositing artifact rather than a genuine defect. To explain this behaviour, the contamination class contains different types of contamination (a piece of cloth, a string or debris) and with only 500 steps and ~ 25 images for this defect type; therefore, the placeholder token [V*] is trained on this inconsistent mix and never converges to a coherent concept, so it basically guesses at inference. The green bottle likely comes entirely from SD2’s pretrained knowledge.

Leather’s generated defects are more promising with poke being the strongest result – a dark, small dot with gradient colour at its orifice, blending the defect naturally with its surrounding. Fold also shows an exposed material underneath, with shadow gradients along the defect. However, it would

have been a better result for cuts. The likely reason is that fold and cut share very similar mask shapes, both are elongated linear regions. With only the mask and a text token to differentiate them, the model may be conflating the two concepts.

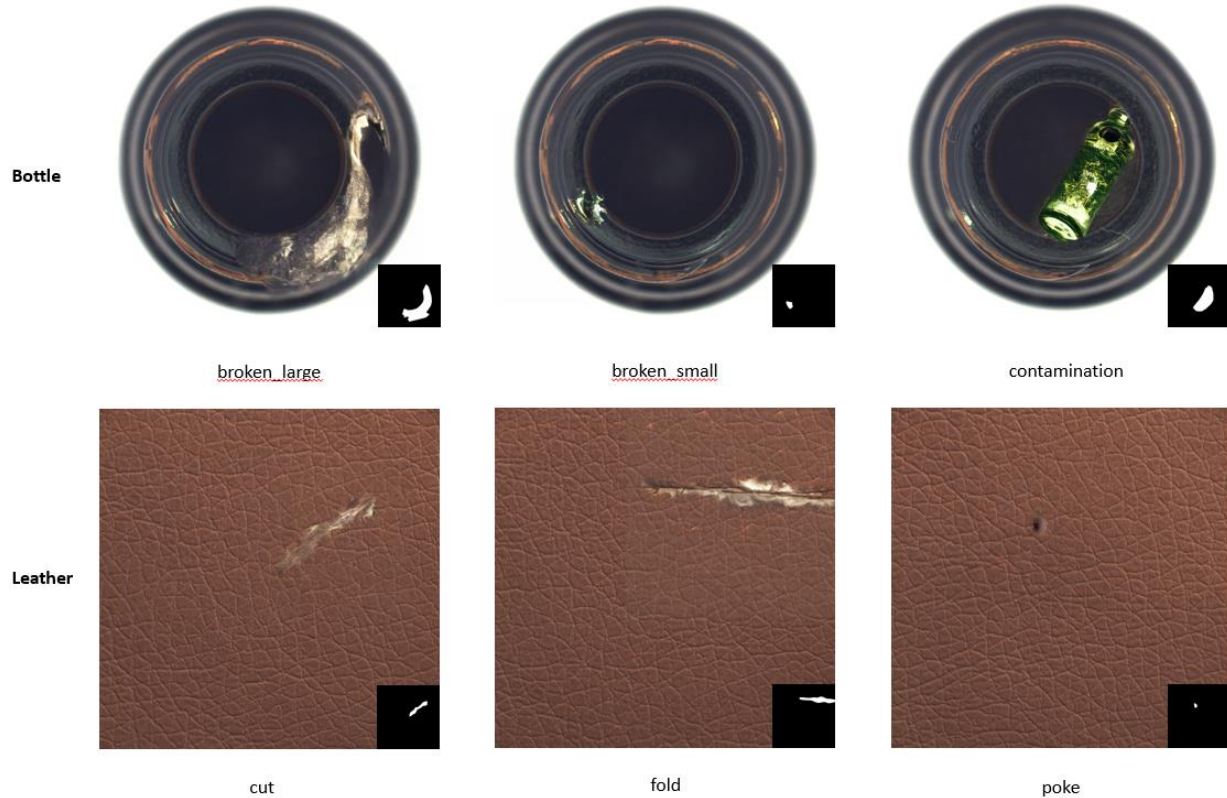


Figure 5. Qualitative results for bottle and leather defects generation.

Quantitative Results

Four metrics were used in this assignment to evaluate how realistic and diverse the generations are:

1. **LPIPS (Learned Perceptual Image Patch Similarity):** measures perceptual similarity between a generated defect image and its paired real defect. Lower = more similar to real defects.
2. **IC-LPIPS (Intra-Cluster LPIPS):** measures diversity among generated images by clustering them against real references and computing pairwise perceptual distances within each cluster. Higher = more varied generations, less mode collapse.
3. **FID (Fréchet Inception Distance):** measures the distributional distance between real and generated image sets in inception feature space. Lower = generated distribution is closer to the real distribution.
4. **KID (Kernel Inception Distance):** similar to FID but uses a kernel-based estimator that is unbiased for small sample sizes, making it more reliable than FID at the scale of this assignment (~25 images per defect). Lower = better.

Although FID requires thousands of generated images, it is still included in this report to provide an additional measurement to evaluate the model's performance across different settings. The authors of DefectFill used KID to evaluate realism and IC-LPIPS for generation diversity. While KID is less sensitive sample size, it is marginally more useful but still considered statistically weak at 25 samples. LPIPS is the most useful metric since it compares realism between the generated and actual sample directly, regardless of the sample size.

Table 1. DefectFill benchmark evaluation

Object	Variation	LPIPS ↓	IC-LPIPS ↑	FID ↓	KID ↓
Bottle	Vanilla	0.28	0.18	137.67	0.06
	Ablated defect	0.29	0.18	143.78	0.07
	Ablated object	0.29	0.17	133.36	0.05
	Ablated attention	0.28	0.17	128.53	0.05
Leather		0.25	0.41	116.52	0.06
Leather (authors)		x	0.30	x	74.66
Bottle (authors)		x	0.12	x	30.99

Comparing the models trained in this assignment, the results presented in Table 1 suggest that the model trained on the leather dataset achieves the best overall performance among the five object categories. It attains the lowest LPIPS score, indicating the highest perceptual similarity to real defects and supporting the observations from the qualitative analysis. Furthermore, the generated defect distribution appears to be both diverse and closely aligned with the real defect distribution, as evidenced by its highest IC-LPIPS score and lowest FID value among all evaluated models.

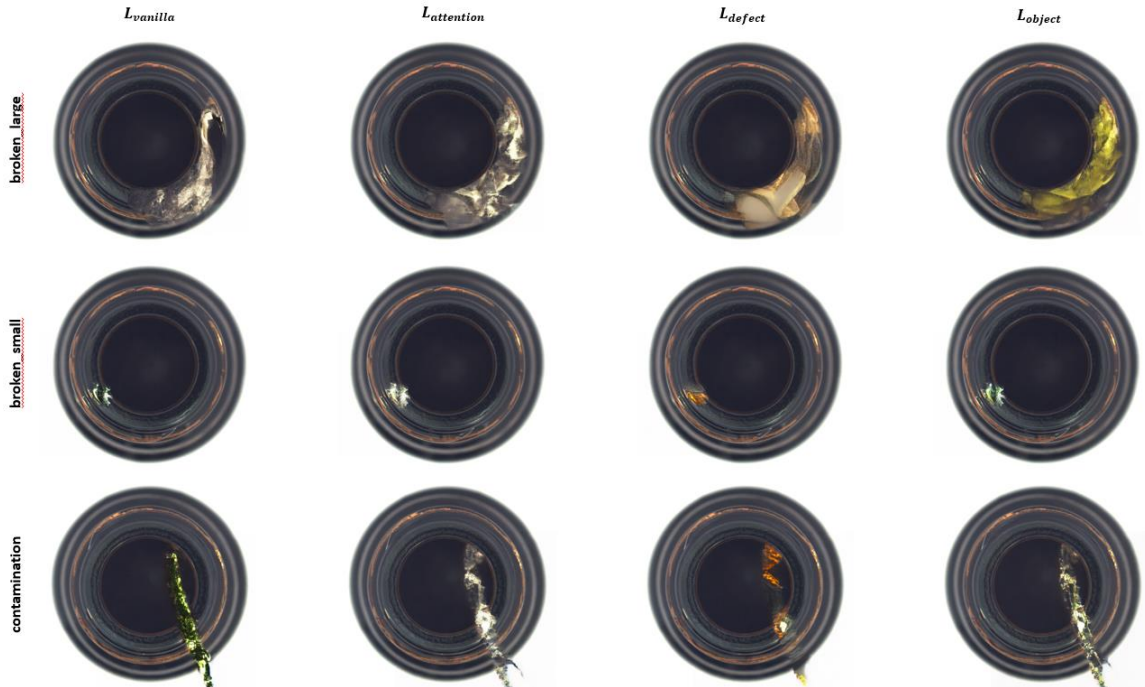


Figure 6. Ablation study: sample generations from 4 different bottle models, each with a type of loss ablated.

For bottle, all four variations perform nearly identically across every metric. The ablations have almost no measurable impact, which suggests either the individual loss components contribute minimally at 500 steps, or the model hasn't trained long enough for the differences to manifest. The ablated attention variant actually achieves the lowest FID (128.53), implying the attention loss may not be helping quality generation at this scale.

Figure 6 illustrates the individual contribution of each loss term to the generated image quality. Compared to the vanilla model, the variant without the defect loss largely loses its ability to generate physically plausible defects, producing only blurry artifacts instead. In contrast, removing the attention loss results in little to no noticeable change in the characteristics of the generated outputs, suggesting that its contribution to visual quality is less pronounced under the current experimental setting. This is consistent with the training curves where attention loss barely converged, confirming that it contributes negligibly to the generation quality at 500 steps with rank-8 LoRA. The defect generated with object loss ablated is much more visible, however, the lack of this term makes the defect look like being pasted directly on the image with no blending.

Based on the reported metrics, my implementation appears to outperform the authors' results numerically. However, I do not believe this reflects the true performance difference between the methods. The absence of an official DefectFill implementation required me to adapt the evaluation pipeline and metric computations without complete knowledge of the authors' exact measurement procedures. Consequently, discrepancies in metric implementation, evaluation settings, or data processing may have contributed to the unexpectedly favorable results obtained in my experiments.

5. Conclusion

This assignment is a proof-of-concept for how inpainting diffusion can generate synthetic defects from just a few samples, following the DefectFill approach. My work extends DefectFill to a multi-defect setting, which introduces additional challenges in distinguishing between defect types that have similar mask characteristics (e.g., cuts and folds in leather). The implementation also adapts an unofficial DefectFill codebase to run in a resource-constrained environment by offloading the VAE and text encoder whenever they are not needed during training and inference.

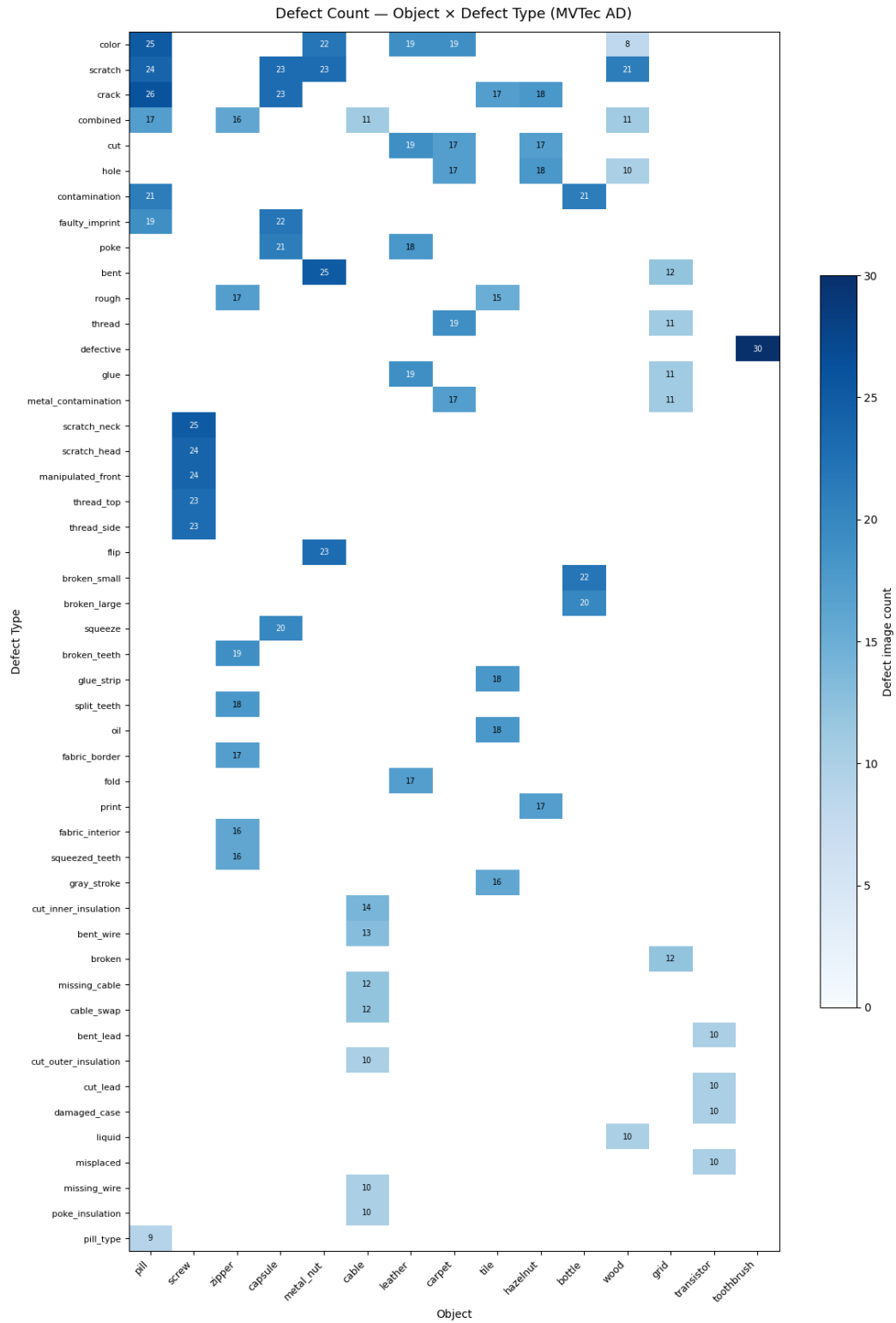
Both qualitative and quantitative analysis show consistent results with leather's generated defects to be the most realistic and diverse. The ablation study also confirms that defect and object losses are the primary drivers of generation quality; removing the object loss leads to unrealistic boundary blending, while removing the defect loss reduces defect generation capability almost entirely. The attention loss, however, contributes negligibly with just 500 training steps, consistent with both the training curves and the qualitative outputs.

References

[1] J. Song, D. Park, K. Baek, S. Lee, J. Choi, E. Kim, and S. Yoon, "DefectFill: Realistic Defect Generation with Inpainting Diffusion Model for Visual Inspection," *arXiv preprint arXiv:2503.13985*, 2025.

- [2] D. Lin, Y. Cao, W. Zhu, and Y. Li, "Few-Shot Defect Segmentation Leveraging Abundant Normal Training Samples Through Normal Background Regularization and Crop-and-Paste Operation," *arXiv preprint arXiv:2007.09438*, 2020.
- [3] Y. Duan, Y. Hong, L. Niu, and L. Zhang, "Few-Shot Defect Image Generation via Defect-Aware Feature Manipulation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 1, pp. 571–578, 2023.
- [4] T. Hu, J. Zhang, R. Yi, Y. Du, X. Chen, L. Liu, Y. Wang, and C. Wang, "AnomalyDiffusion: Few-Shot Anomaly Image Generation with Diffusion Model," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 8, pp. 7472–7480, 2024.
- [5] R. Gal, O. Patashnik, H. Maron, G. Chechik, and D. Cohen-Or, "An Image is Worth One Word: Personalizing Text-to-Image Generation Using Textual Inversion," *arXiv preprint arXiv:2208.01618*, 2022.
- [6] A. Sigurdsson, "DefectFill," GitHub repository. [Online]. Available: <https://github.com/axelsig1/DefectFill>. [Accessed: May 31, 2026].

Appendix



A1. Defect type distribution across different objects (MVTec AD Dataset)